

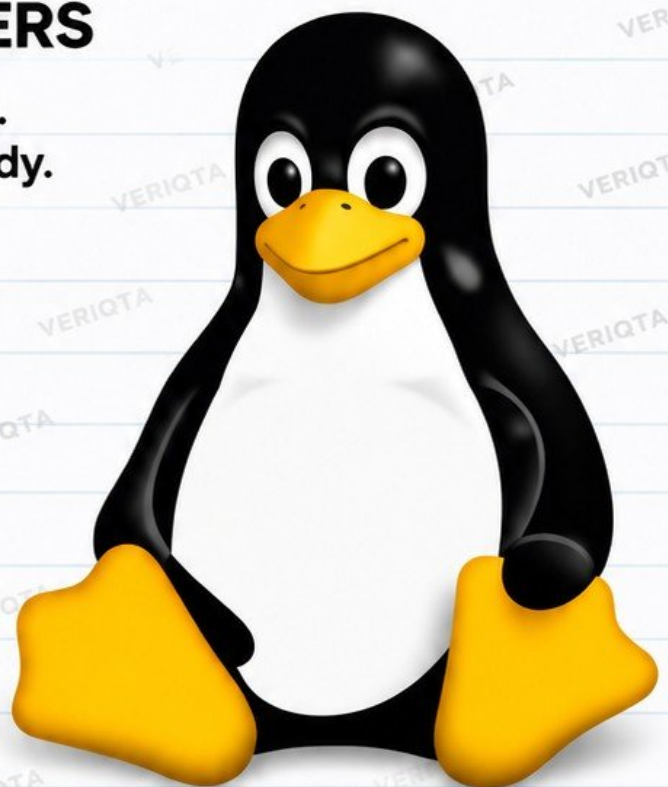
LINUX

POCKET GUIDE

ESSENTIAL LINUX KNOWLEDGE FOR JUNIOR ENGINEERS

Simple Notes. Real Commands.
Practical Skills. Production Ready.

- ✓ Commands
- ✓ Examples
- ✓ Diagrams
- ✓ Troubleshooting
- ✓ Real DevOps Context
- ✓ Beginner Friendly



**LEARN
COMMANDS**



**BUILD
SKILLS**



**SOLVE
PROBLEMS**



**APPLY
IN DEVOPS**

YOUR LINUX JOURNEY STARTS HERE

WHAT IS LINUX?



- Linux is a free and open source operating system.
- It allows you to run applications, manage hardware and work with files.
- It is stable, secure and widely used in servers, cloud, containers and DevOps environments.

KERNEL vs OPERATING SYSTEM

KERNEL

- Core of the operating system.
- Manages hardware (CPU, memory, storage, devices).
- Provides services to applications.

OPERATING SYSTEM

- Kernel + system tools + libraries + user interface.
- Examples: Linux, Windows, macOS.
- Linux is an operating system that uses the Linux kernel.

DISTRIBUTION (DISTRO)



- A Linux distribution (distro) is a complete version of Linux.
- It includes the kernel, system tools, package manager and default applications.
- Distro makes Linux easy to install and use.

SHELL



- A shell is a command line interpreter.
- It takes your commands and tells the operating system what to do.
- Common shells: Bash (default), Zsh, Fish, etc.

TERMINAL



- A terminal is the program you use to interact with the shell.
- You type commands in the terminal.
- It gives you a text based interface to control Linux.

COMMON LINUX DISTRIBUTIONS



Ubuntu Beginner friendly. Widely used.



CentOS Popular in enterprise environments.



Red Hat Enterprise Linux (RHEL)
Common in businesses.



Debian Stable and reliable.



openSUSE Used in enterprise and development.

Others: Fedora, Amazon Linux, AlmaLinux, Rocky Linux, Kali Linux.

WHY LINUX DOMINATES



- Free and open source.
- Highly stable and secure.
- Runs on most servers in the world.
- The backbone of cloud platforms (AWS, Azure, Google Cloud).
- Essential for containers (Docker, Kubernetes).
- Powerful for DevOps, automation and infrastructure engineering.
- Large community and constant development.

LINUX MENTAL MODEL



JUNIOR ENGINEER TIP

Learn to use the terminal early. It gives you more control, works on all Linux servers, and is essential for DevOps.



REMEMBER THIS

Linux is everywhere — servers, cloud, containers, Kubernetes, CI/CD, and modern infrastructure.



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

The Linux filesystem is a single hierarchical tree. Everything starts from / (root). Each directory has a specific purpose.

DIRECTORY	WHAT IT CONTAINS / PURPOSE	EXAMPLES / NOTES
 /	Root of the entire filesystem. All files and directories are under this.	Top level. Everything starts here.
 /home	User home directories.	/home/username
 /etc	System configuration files.	/etc/nginx/ /etc/ssh/
 /var	Variable data such as logs, databases, spools and application data.	/var/log/ /var/www/
 /tmp	Temporary files. Usually cleared on reboot.	Used by applications for temporary data.
 /usr	User applications and read-only data. Contains most of the installed software.	/usr/bin/ /usr/lib/
 /opt	Optional or third-party software. Used for custom applications.	/opt/<app-name>/
 /bin	Essential user commands. Basic tools needed for all users.	ls, cp, mv, cat, bash
 /sbin	System administration commands. Usually used by root or with sudo.	ip, reboot, fdisk, shutdown
 /dev	Device files. Represents hardware devices as files.	/dev/sda (disk) /dev/null (null device)
 /proc	Virtual filesystem with process and kernel information.	/proc/cpuinfo /proc/meminfo
 /sys	Virtual filesystem for kernel and hardware information.	/sys/class/ /sys/devices/



KEY THINGS TO REMEMBER

- Linux uses a single tree structure starting from /.
- Configurations are usually in /etc.
- Logs are usually in /var/log/.
- User files are in /home/username/.
- Temporary files are in /tmp.
- Most system tools are in /usr, /bin and /sbin.



JUNIOR ENGINEER TIP

- ✓ Use `ls -l /` to explore the top level directories.
- ✓ Use `cd` and `ls` to look inside any directory.
- ✓ Learn these directories. You will use them every day.
- ✓ When troubleshooting, check /etc, /var/log, and /proc first.



WARNING

Do not delete or modify system files unless you understand what they do. Changing files in /etc, /bin, /sbin, /dev, /proc or /sys can break your system.



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Linux has a directory structure. These commands help you move around, view, and create directories so you can work with files and manage your system.

pwd – Print Working Directory

Shows the full path of your current directory.

```
$ pwd
/home/veriqta
```

ls – List Files and Directories

Lists the files and directories in the current directory.

```
$ ls
Documents Downloads file.txt
Pictures Scripts
```

cd – Change Directory

Moves you to a different directory.

```
$ cd Documents
$ pwd
/home/veriqta/Documents
```

tree – Display Directory Tree

Shows a visual tree of files and directories.

```
$ tree
.
├── Documents
├── Downloads
├── Pictures
└── file.txt
4 directories, 1 file
```

mkdir – Make Directory

Creates a new directory.

```
$ mkdir projects
$ ls
Documents projects Scripts
```

Absolute vs Relative Paths

- Absolute path: Full path from / (root).

```
/home/veriqta/Documents
```

- Relative path: Path relative to your current directory.

```
Documents/file.txt
```

., .. and ~

- . Current directory
- .. Parent directory (one level up)
- ~ Home directory (shortcut)

```
$ cd .. # go up one level
$ cd . # stay in current directory
$ cd ~ # go to home directory
```

Hidden Files

Files starting with . are hidden.

```
$ ls
file.txt Documents
$ ls -a
.  ..  .bashrc  .config
file.txt Documents
```

Use `ls -a` to show hidden files.

Common Options for ls

Useful flags:

-l	Long format (details)
-a	Show hidden files
-h	Human readable file sizes
-t	Sort by modified time
-r	Reverse order



BEGINNER MISTAKE

Running commands from the wrong directory. This can cause you to edit, delete, or create files in the wrong location.



BEST PRACTICES

- ✓ Always check your current directory with `pwd`.
- ✓ Use `ls` to see what's in the directory.
- ✓ Be careful with paths before running destructive commands.
- ✓ Use tab completion to avoid typing mistakes.
- ✓ Take your time, especially in production servers.



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

These commands help you create, copy, move, rename, locate and delete files. You will use them every day when working on Linux servers.

touch – Create a File

Creates an empty file. If the file exists, it updates the timestamp.

```
$ touch file.txt
$ touch /tmp/notes.txt
```



Example: Create multiple files

```
$ touch file1.txt file2.txt file3.txt
```

cp – Copy Files or Directories

Copies files or directories to a new location.

```
$ cp file.txt backup.txt
$ cp -r project/ project_backup/
```



Example: Copy a directory

```
$ cp -r myfolder/ /tmp/
```

mv – Move or Rename

Moves files to a different location or renames them.

```
$ mv file.txt /tmp/
$ mv oldname.txt newname.txt
```



Example: Rename a file

```
$ mv report.txt report-2026.txt
```

rm – Delete Files or Directories

Deletes files or directories permanently.

```
$ rm file.txt
$ rm -r myfolder/
```



Be Careful!

rm deletes files permanently. There is no recycle bin.

Use **-i** for confirmation:

```
$ rm -i file.txt
```

file – Check File Type

Shows the type of a file (text, binary, image, etc.).

```
$ file file.txt
$ file image.jpg
$ file script.sh
```



Example Output:

```
file.txt: ASCII text
image.jpg: JPEG image data
script.sh: Bourne-Again shell script
```

stat – File Details

Shows detailed information about a file (size, permissions, timestamps, etc.).

```
$ stat file.txt
```



Example Output (shortened):

```
File: file.txt
Size: 123 bytes
Modify: 2026-09-21 10:30:45
Access: 2026-09-21 10:31:12
Permissions: -rw-r--r--
```

find – Locate Files

Searches for files and directories based on name, type, size, etc.

```
$ find / -name file.txt
$ find . -name "*.log"
$ find /var -type d -name nginx
```



Example: Find all .txt files in current directory

```
$ find . -name "*.txt"
```

Common File Operations

Create a file	touch file.txt
Copy a file	cp file.txt backup.txt
Copy a directory	cp -r dir/ dir_backup/
Move a file	mv file.txt /tmp/
Rename a file	mv old.txt new.txt
Delete a file	rm file.txt
Delete a directory	rm -r dir/
Check file type	file file.txt
File details	stat file.txt
Find files	find / -name file.txt

Best Practices

- ✓ Always double-check the file path before running **rm**.
- ✓ Use **rm -i** for important files.
- ✓ Use descriptive file names.
- ✓ Use **find** to locate files instead of guessing.
- ✓ Be careful when working in **/**, **/etc**, **/var** or system directories.
- ✓ Check file type and details when troubleshooting.

Common Beginner Mistakes

- ✗ Running **rm** on the wrong file or directory.
- ✗ Forgetting **-r** when deleting a directory.
- ✗ Using **rm** without **-i** (no confirmation).
- ✗ Copying files to the wrong location.
- ✗ Overwriting files without backup.
- ✗ Not checking the current directory (**pwd**).

“ CHECK THE PATH. THINK BEFORE YOU DELETE.”



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Linux provides powerful commands to read, view and search text files. You will use these commands daily to read configuration files, view logs and troubleshoot issues on servers.

cat – Display File Content

Shows the full content of a file.

```
$ cat file.txt
$ cat /etc/nginx/nginx.conf
```

Good for small files.
Not ideal for very large files.

less – View File Page by Page

Opens a file and lets you scroll page by page.

```
$ less file.txt
$ less /var/log/syslog
```

Use arrow keys to scroll.
Press q to exit.

head – Show First Lines

Shows the first 10 lines by default.

```
$ head file.txt
$ head -n 20 file.txt
```

Use -n to specify number of lines.
Example: head -n 50 access.log

tail – Show Last Lines

Shows the last 10 lines by default.

```
$ tail file.txt
$ tail -n 20 /var/log/syslog
```

Use -n to specify number of lines.

tail -f – Follow a File in Real Time

Continuously shows new lines as they are added.

```
$ tail -f /var/log/nginx/access.log
$ tail -f /var/log/app.log
```

Press Ctrl + C to stop.

wc – Count Lines, Words, Bytes

Shows the number of lines, words and bytes in a file.

```
$ wc file.txt
$ wc -l file.txt # lines
$ wc -w file.txt # words
$ wc -c file.txt # bytes
```

Useful for quick file analysis.

grep – Search for Text

Searches for a pattern in a file.

```
$ grep error app.log
$ grep -i error app.log # ignore case
$ grep -n error app.log # show line no.
$ grep -r error /var/log # search in dir
```

Use -i for case insensitive search.
Use -n to show line numbers.
Use -r to search recursively in directories.

PRACTICAL EXAMPLE

Following an application log during an incident

```
$ tail -f /var/log/app.log
2026-09-21 10:15:23 INFO User lo
2026-09-21 10:15:45 ERROR Connec
2026-09-21 10:16:02 ERROR Timeou
2026-09-21 10:16:10 INFO Retrying
```

This shows new log entries in real time so you can see errors as they happen.

COMMON LOG FILES

/var/log/syslog	System logs
/var/log/auth.log	Authentication logs
/var/log/nginx/	Nginx logs
/var/log/apache2/	Apache logs
/var/log/app.log	Application logs
/var/log/messages	System messages
/var/log/kern.log	Kernel logs



IMPORTANT

- Log files can be very large.
- Use **less** or **tail** instead of **cat** for large files.
- Be careful with sensitive information in logs (passwords, tokens, etc.).



QUICK TIPS

- Use **less** for manual reading.
- Use **tail -f** for real-time monitoring.
- Use **grep** to find errors or specific patterns.
- Combine commands: **tail -f app.log | grep error**



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Pipes and redirection let you connect commands, control input and output, and handle errors. They are essential for automation, log analysis and troubleshooting in real Linux environments.

| – Pipe (Pass Output)

Sends the output of one command to another command.

```
$ ls | grep txt
$ ps aux | grep nginx
```

The first command's stdout becomes the second command's stdin.

> – Redirect Output (Overwrite)

Sends output to a file (overwrites existing content).

```
$ ls > files.txt
$ echo "Hello" > test.txt
```

Creates the file if it does not exist. Overwrites if it exists.

>> – Redirect Output (Append)

Sends output to a file (appends without overwriting).

```
$ ls >> files.txt
$ echo "New line" >> test.txt
```

Useful for logs and continuous output.

< – Redirect Input

Uses a file as input for a command (instead of keyboard).

```
$ wc -l < file.txt
$ sort < names.txt
```

The file becomes the command's stdin.

2> – Redirect Error Output

Sends error messages (stderr) to a file.

```
$ ls /invalid 2> error.log
$ grep error app.log 2> grep.err
```

Only errors are written to the file.

2>&1 – Redirect Errors and Output

Sends both stdout and stderr to the same file.

```
$ command > output.log 2>&1
$ ls /invalid > all.log 2>&1
```

Useful for complete logs.

stdin, stdout, stderr

Every Linux command has 3 streams:

stdin	0	Standard input (usually keyboard or a file)
stdout	1	Standard output (normal output)
stderr	2	Standard error (error messages)

You can redirect these streams to files, other commands or `/dev/null`.

Practical Example

Follow Nginx logs and filter errors in real time.

```
$ journalctl -u nginx | grep error
Sep 21 10:15:23 server nginx[123]:
error: upstream timed out
Sep 21 10:16:01 server nginx[123]:
error: connect() failed
```

This shows only error lines from the Nginx service logs.

Why Command Composition Matters

- ✓ Lets you combine simple tools to solve complex problems.
- ✓ Helps in log analysis and incident investigation.
- ✓ Reduces the need for scripts in many situations.
- ✓ Makes automation more powerful.
- ✓ Saves time and improves productivity.
- ✓ Essential skill for DevOps, SRE and system administration.



IMPORTANT

- Be careful when redirecting output to existing files.
- Check your commands before running them.
- Use `2>&1` when you need both normal output and errors.
- You can discard unwanted output using `/dev/null` (e.g. command `> /dev/null 2>&1`).



QUICK TIPS

- Use pipes to filter and search.
- Use `>` to overwrite and `>>` to append.
- Use `2>` for errors and `2>&1` for both.
- Think about where each command's output goes.
- Practice combining commands — it will make you faster and more efficient.



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Linux uses users and groups to control access, permissions and security. Understanding identity is essential for system administration, DevOps and troubleshooting.

whoami – Current User

Shows the username of the current user.

```
$ whoami
veriqta
```

Useful to confirm which user you are logged in as.

id – User and Group IDs

Shows your user ID (UID), group ID (GID) and all groups you belong to.

```
$ id
uid=1000(veriqta) gid=1000(veriqta)
groups=1000(veriqta),27(sudo)
```

UID identifies the user.
GID identifies the primary group.

who – Logged In Users

Shows who is currently logged in to the system.

```
$ who
veriqta pts/0 2026-09-21 10:15
root pts/1 2026-09-21 09:02
```

Shows username, terminal and login time.

groups – Your Groups

Shows the groups you belong to.

```
$ groups
veriqta sudo docker
```

A user can belong to multiple groups.

/etc/passwd – User Database

Stores user account information on the system.

```
$ cat /etc/passwd | head -3
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sbin/nologin
veriqta:x:1000:1000:/home/veriqta:/bin/bash
```

Fields: username:x:UID:GID:comment:home:shell

/etc/group – Group Database

Stores group information.

```
$ cat /etc/group | head -3
root:x:0:
daemon:x:1:
sudo:x:27:veriqta
```

Fields: groupname:x:GID:members

useradd – Create a User

Creates a new user account.

```
$ sudo useradd devuser
$ sudo useradd -m -s /bin/bash appuser
```

Use **-m** to create a home directory.
Use **-s** to specify the login shell.

usermod – Modify a User

Modifies an existing user account.

```
$ sudo usermod -aG docker veriqta
$ sudo usermod -s /bin/bash devuser
```

Use **-aG** to add a user to a group.
Use **-s** to change the login shell.

User Types

Regular user	Normal user for daily work.
Root user	UID 0. Full administrative access. Use with care.
Service account	Used by applications and services (e.g. nginx, mysql).
System user	Low UID (usually < 1000). Created by the system.

User IDs and Group IDs

- Each user has a unique **User ID (UID)**.
- Each group has a unique **Group ID (GID)**.
- UID 0 is the **root** user.
- Regular users usually start from **UID 1000**.
- Groups control access to files, directories and system resources.

Best Practices

- ✓ Use least privilege.
- ✓ Do not use **root** for daily work.
- ✓ Use groups to manage access.
- ✓ Create dedicated service accounts for applications.
- ✓ Regularly review user and group membership.
- ✓ Remove unused accounts.
- ✓ Use strong authentication methods (e.g. SSH keys).

Important

- The root user has full control of the system. A mistake can break your server.
- Do not share user accounts.
- Remove unnecessary users and groups.
- Monitor service accounts.
- Check **/etc/passwd** and **/etc/group** when troubleshooting identity issues.



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Linux permissions control who can read, write and execute files and directories. Understanding permissions is essential for security, system administration and DevOps.

Read, Write, Execute

Every file and directory has three basic permissions.

Permission	Symbol	Meaning
Read	r	View file content (list directory)
Write	w	Modify file content (create, delete, rename)
Execute	x	Run a file (enter a directory)

Owner, Group, Others

Each file has three permission sets.

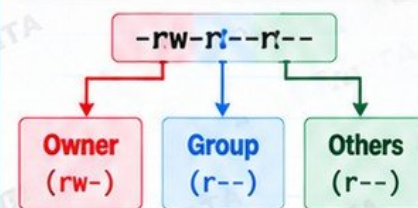
Category	Who it is
Owner (u)	The user who owns the file.
Group (g)	Users in the file's group.
Others (o)	All other users.

Permissions are shown in this order:
owner group others
(rwx) (rwx) (rwx)

View Permissions with ls -l

Use `ls -l` to see file permissions.

```
$ ls -l file.txt
-rw-r--r-- 1 veriqta veriqta 123
Sep 21 10:00 file.txt
```



Change Permissions with chmod

Use `chmod` to change file permissions.

```
$ chmod 755 script.sh
$ chmod 644 file.txt
```

Syntax: `chmod [permissions] [file]`
Example: `chmod 755 script.sh`

Change Owner with chown

Use `chown` to change the file owner.

```
$ sudo chown veriqta file.txt
$ sudo chown veriqta:dev file.txt
```

Syntax: `chown [user] [file]`
Syntax: `chown [user:group] [file]`

Change Group with chgrp

Use `chgrp` to change the file group.

```
$ sudo chgrp dev file.txt
$ sudo chgrp -R dev /project
```

Syntax: `chgrp [group] [file]`
Use `-R` for directories.
Example: `chgrp -R dev /project`

Numeric Permissions

Permissions can also be set using numbers.

Number	Permission	Meaning
0	---	No permission
1	--x	Execute only
2	-w-	Write only
4	r--	Read only
6	rw-	Read and write
7	rwx	Read, write and execute

Example:

```
chmod 644 file.txt (rw-r--r--)
chmod 600 file.txt (rw-----)
chmod 755 script.sh (rwxr-xr-x)
```

Common Examples

Some common permission combinations.

Permission	Symbol	Use Case
644	rw-r--r--	Regular file (e.g. config)
600	rw-----	Sensitive file (e.g. private key)
755	rwxr-xr-x	Executable script (e.g. .sh file)
777	rwxrwxrwx	All permissions (avoid using)

Why chmod 777 is Usually Wrong



- Gives full access to everyone.
- Creates a serious security risk.
- Can be exploited by malicious users or processes.
- Masks the real problem instead of fixing it.
- Use the minimum permissions needed.

Least Privilege

- ✓ Only give the permissions that are needed.
- ✓ Reduces security risk.
- ✓ Prevents accidental or malicious changes.
- ✓ Follow least privilege in all environments (servers, cloud, containers, etc.).
- ✓ Review permissions regularly.



“GIVE ONLY THE ACCESS THAT IS NEEDED, NOT MORE.”



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Understanding sudo, root and privilege is essential for safe system administration. It helps you perform administrative tasks without putting your system at unnecessary risk.

What is sudo?

Runs a command with elevated privileges (as another user, usually root).

```
$ sudo ls /etc
$ sudo systemctl restart nginx
$ sudo apt update
```

Your normal user runs the command, but with temporary elevated privileges.

What is su?

Switches to another user account (often root).

```
$ su -
# enter root password
# now you are root
# cd /etc
# exit
```

Use **su -** to get a full root shell. Type **exit** to return to your user.

Root Privileges

The root user has full control over the system.

```
$ whoami
veriqta
$ sudo whoami
root
```

Root can read, write, delete and change anything on the system. With great power comes great risk.

/etc/sudoers

Controls which users can run commands with sudo.

```
$ sudo visudo
```



Always use **visudo** to edit this file. Do not edit **/etc/sudoers** directly. A syntax error can lock you out of sudo access.

Example /etc/sudoers Entry

Allows a user to run commands with sudo.

```
# Allow veriqta to run any command
veriqta ALL=(ALL) ALL
```

Format:

username host=(user) commands

In this example, user veriqta can run any command as any user on any host.

Why Privileged Access Exists

Some tasks require elevated privileges because they affect the whole system.

- ✓ Install and update software
- ✓ Manage system services
- ✓ Change system configuration
- ✓ Access hardware and network settings
- ✓ Perform system maintenance
- ✓ Manage users and groups

These tasks require more than normal user permissions.

Safe Administration Practices

- ✓ Use **sudo** instead of switching to root.
- ✓ Only run privileged commands when needed.
- ✓ Be careful with powerful commands (e.g. **rm**, **chmod**, **systemctl**).
- ✓ Use specific commands instead of broad access.
- ✓ Keep an audit trail of changes.
- ✓ Follow your organization's access policies.

Why You Should Not Run Everything with sudo

- ✗ Increases the risk of accidental damage.
- ✗ Can hide permission and configuration problems.
- ✗ Makes it easier for malware to cause harm.
- ✗ Reduces accountability.
- ✗ Can lead to security breaches.
- ✗ You might overwrite important system files without realizing it.

Use the least privilege needed to complete the task.

Key Takeaways

- ✓ **sudo** gives temporary elevated privileges.
- ✓ **su** switches to another user (often root).
- ✓ Root has full control of the system.
- ✓ **/etc/sudoers** defines who can use sudo.
- ✓ Privileged access exists for system administration tasks.
- ✓ Always follow safe administration practices.
- ✓ Do not run everything with **sudo**.
- ✓ Use the least privilege needed.

“POWERFUL ACCESS SHOULD BE USED WISELY, NOT CONSTANTLY.”



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Linux runs multiple processes at the same time. Understanding processes and jobs helps you monitor system activity, troubleshoot issues and manage resources effectively.

PID and PPID

Each process has a unique Process ID (PID) and a Parent Process ID (PPID).

```
$ ps -ef | head
UID      PID    PPID  CMD
Root      1      0    /sbin/init
root     234      1    /usr/bin/sshd
veriqta 1050    234    bash
```

PID identifies the process.
PPID identifies the parent process that started it.

Foreground vs Background

Foreground: runs in the terminal and blocks it.

Background: runs without blocking the terminal.

```
$ sleep 60 # foreground
$ sleep 60 & # background
[1] 1234
```

Use **&** to run a process in the background.

ps – View Processes

Shows information about running processes.

```
$ ps aux | head
USER      PID %CPU %MEM    COMMAND
root         1  0.0  0.1    /sbin/init
veriqta 1050  0.0  0.1      bash
veriqta 1123  0.0  0.2     nginx
```

Common options:

```
ps aux      # all processes
ps -ef      # full format
ps -p <PID> # specific process
```

top – Real Time Monitoring

Shows live system processes.

```
$ top
PID USER %CPU %MEM COMMAND
1 root 0.0 0.1 init
1050 veriqta 0.0 0.1 bash
1123 veriqta 0.3 1.2 nginx
```

Press **q** to exit.

pgrep – Find Processes

Searches for processes by name.

```
$ pgrep nginx
1123
1130
```

Add **-l** to show the process name.
Example: `pgrep -l nginx`

jobs – List Background Jobs

Shows your background jobs in the current terminal.

```
$ sleep 300 &
[1] 2345
$ jobs
[1] Running      sleep 300
```

Use **fg %1** to bring a job to the foreground.
Use **bg %1** to continue it in the background.

kill – Terminate a Process

Sends a signal to a process by PID.

```
$ kill 1234 # default SIGTERM
$ kill -9 1234 # force kill (SIGKILL)
```

Try a normal kill first. Use **-9** only when the process does not stop.

pkill – Kill by Name

Terminates processes by name.

```
$ pkill nginx # kill all nginx
$ pkill -f myapp.py # kill by full name
```

Use with care. It can kill multiple processes at once.

Signals

Signals tell a process what to do.

Signal	Number	Meaning
SIGTERM	15	Graceful termination
SIGKILL	9	Force kill (cannot be ignored)
SIGHUP	1	Reload configuration
SIGINT	2	Interrupt (Ctrl + C)
SIGSTOP	19	Stop (cannot be ignored)
SIGCONT	18	Continue a stopped process

Diagnosing a Runaway Process

Find and investigate high CPU or memory usage.

```
$ top # find the process
$ ps -p 1234 -o pid,ppid,cmd # check details
$ kill 1234 # stop it
```

Look for processes using high CPU or memory and decide if it is safe to stop them.



IMPORTANT

- Do not kill system processes unless you understand the impact.
- Use SIGTERM before SIGKILL.
- Be careful with **pkill** (it can kill multiple processes).
- A runaway process can affect system performance.
- Always investigate the cause.

Key Takeaways

- ✓ Each process has a PID and PPID.
- ✓ Use **ps**, **top**, **pgrep**, **jobs**, **kill** and **pkill**.
- ✓ Understand foreground vs background.
- ✓ Use signals appropriately.
- ✓ Investigate runaway processes.
- ✓ Do not force kill unless necessary.
- ✓ Good process management keeps your systems stable.
- ✓ This is a critical skill for DevOps, SRE and system administration.

“ A WELL MANAGED SYSTEM KEEPS RUNNING. KNOW YOUR PROCESSES.”



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

systemd is the default init system on most modern Linux distributions. It manages services, daemons, system startup and more. Understanding systemd is essential for server administration, DevOps and troubleshooting.

Services and Daemons

A service is a managed application.
A daemon is a background process that usually runs as a service.

Service	Managed by systemd (unit file).
Daemon	Background process (usually no user interface).
systemd	Starts, stops, monitors and manages services.

Examples: nginx, sshd, docker, cron, mariadb.

Check Service Status

Shows the current status of a service.

```
$ systemctl status nginx
● nginx.service - A high performance web server
   Loaded: loaded (/usr/lib/systemd/
   Unit: enabled
   Active: active (running)
   Main PID: 1123 (nginx)
```

Key states:

- active (running) – service is running
- inactive (dead) – service is stopped
- failed – service has an error
- activating – service is starting
- deactivating – service is stopping

Start, Stop, Restart

Control services using systemctl.

```
$ systemctl start nginx    # start
$ systemctl stop nginx     # stop
$ systemctl restart nginx  # restart
$ systemctl reload nginx   # reload
```

Use restart to apply configuration changes. Use reload when supported (it is faster and does not fully stop the service).

Enable and Disable

Control whether a service starts automatically at boot.

```
$ systemctl enable nginx  # start at boot
$ systemctl disable nginx # do not start
$ systemctl is-enabled nginx
```

Units in systemd

systemd manages different types of units.

Unit Type	Description
.service	Application service (most common)
.socket	Socket activation
.target	Group of units (like runlevels)
.timer	Scheduled tasks (like cron)
.mount	File system mount points
.path	Triggered by file system changes
.slice	Resource management
.scope	External processes

Service Files

Service configuration files are stored in:

```
/etc/systemd/system/ # custom services
/usr/lib/systemd/system/ # distro services
```

Example: /etc/systemd/system/nginx.service

A typical service file:

```
[Unit]
Description=NGINX Web Server
After=network.target

[Service]
ExecStart=/usr/sbin/nginx
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Service Startup

How services start during boot.

- 1 systemd reads unit files.
- 2 Units are started based on dependencies (e.g. network.target).
- 3 Enabled services start automatically at boot.
- 4 You can check the boot process with:

```
$ systemd-analyze blame
$ systemd-analyze critical-chain
```

Use targets to control groups of services (e.g. multi-user.target for normal boot).

Troubleshooting a Failed Service

Steps to diagnose and fix a service issue.

- 1 Check status: `systemctl status <service>`
- 2 View logs: `journalctl -u <service>`
- 3 Check configuration: `systemctl cat <service>`
- 4 Test configuration: `<service> -t` (if supported)
- 5 Look for errors: `journalctl -xe`
- 6 Fix the issue and restart: `systemctl restart <service>`



Common Issues

- Service fails to start due to config errors.
- Missing dependencies.
- Port already in use.
- Wrong file permissions.
- Service not enabled at boot.
- Application errors (check logs).
- Use `journalctl -u <service>` to see detailed error messages.

Key Takeaways

- ✓ systemd manages services and daemons.
- ✓ Use `systemctl` to control services.
- ✓ Understand unit files and service configuration.
- ✓ Enable services that must start at boot.
- ✓ Use logs to troubleshoot issues.
- ✓ Learn to read service status and error messages.
- ✓ Practice troubleshooting failed services in a safe environment.
- ✓ Essential skill for DevOps, SRE and system administration.

“STABLE SYSTEMS RUN ON WELL MANAGED SERVICES.”



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Logs show what happens on your system. They are critical for troubleshooting, security, compliance and incident response. Learn how to read and search logs before making any changes.

Why Logs Matter

- ✓ Help you understand what happened.
- ✓ Show errors, warnings and important events.
- ✓ Are essential for troubleshooting.
- ✓ Provide evidence for security investigations.
- ✓ Help you avoid guessing and make informed decisions.

“Logs tell the truth. Always check the logs before making changes.”

/var/log Directory

Traditional log files are stored in `/var/log`.

```
$ ls -l /var/log
drwxr-xr-x 2 root root 4096 Sep 21 10:00
auth.log
syslog
kern.log
nginx/
apache2/
mysql/
messages
secure
```

Different applications store logs in files under `/var/log`. The exact files depends on your Linux distribution.

Systemd Journal

Modern Linux distributions use systemd journal to collect and manage logs.

```
$ journalctl
Sep 21 09:45:01 veriqta systemd[1]:
Started nginx.service.
Sep 21 09:45:02 veriqta nginx[1123]:
nginx: the configuration file
/etc/nginx/nginx.conf syntax is ok
```

The journal stores logs from the kernel, services and applications. Use `journalctl` to view and search logs.

journalctl – Basic Usage

View logs from the systemd journal.

```
$ journalctl          # show all logs
$ journalctl -n 50    # last 50 lines
$ journalctl --no-pager # no page by page
```

Press `q` to exit. Use `--no-pager` for direct output (useful in scripts).

journalctl -u – Specific Service

View logs for a specific service.

```
$ journalctl -u nginx
$ journalctl -u nginx -n 50
$ journalctl -u ssh -f
```

Use `-u` followed by the service name. Examples: `nginx`, `ssh`, `docker`, `mysql`.

journalctl -f – Follow Logs

Follow logs in real time (like `tail -f`).

```
$ journalctl -f          # follow all logs
$ journalctl -u nginx -f # follow nginx
$ journalctl -u ssh -f   # follow ssh
```

Keep running and shows new log entries as they happen. Press `Ctrl + C` to stop.

journalctl --since – Time Filter

View logs from a specific time.

```
$ journalctl --since "1 hour ago"
$ journalctl --since "2026-09-21 08:00"
$ journalctl -u nginx --since "today"
```

You can also use `--until` to set an end time. Examples: `"yesterday"`, `"2 days ago"`.

Application vs System Logs

Type	Examples	Description
System logs	<code>/var/log/syslog</code> <code>/var/log/messages</code> <code>journalctl</code>	System events, kernel, services, boot, errors.
Application logs	<code>/var/log/nginx/</code> <code>/var/log/apache2/</code> <code>/var/log/mysql/</code>	Application specific logs (web servers, databases, etc.).
Security logs	<code>/var/log/auth.log</code> <code>/var/log/secure</code>	Authentication, login attempts, security events.

Finding Evidence Before Changing Anything

- ✓ Always check the logs first.
- ✓ Understand the timeline of events.
- ✓ Look for errors, warnings and unusual activity.
- ✓ Identify the root cause.
- ✓ Only make changes after you understand what is happening.
- ✓ Keep logs for incident reports.

“Investigate. Understand. Then change.”

Common journalctl Options

Option	Description
<code>-u</code>	Show logs for a specific service
<code>-f</code>	Follow logs in real time
<code>-n <number></code>	Show last N lines
<code>--since</code>	Show logs since a specific time
<code>--until</code>	Show logs until a specific time
<code>--no-pager</code>	Disable pagination
<code>-p <level></code>	Filter by priority (err, warning, info, etc.)

Log Priorities (Levels)

Level	Number	Meaning
emerg	0	System is unusable
alert	1	Action must be taken immediately
crit	2	Critical conditions
err	3	Error conditions
warning	4	Warning conditions
notice	5	Normal but significant
info	6	Informational messages
debug	7	Debug-level messages

Key Takeaways

- ✓ Logs are your best friend during troubleshooting.
- ✓ Use `journalctl` and `/var/log` effectively.
- ✓ Check logs before making any changes.
- ✓ Know the difference between system and application logs.
- ✓ Use time filters to find relevant events.
- ✓ Logs help with security, compliance and incident investigations.



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Package managers help you install, update, remove and manage software. Understanding packages, repositories and dependencies is essential for system administration, DevOps and secure operations.

What are Packages?

A package is a compressed file that contains software, configuration files, libraries and metadata.

Examples:

- nginx (web server)
- git (version control)
- python (programming language)
- vim (text editor)

Packages make it easy to install, update and remove software.

What are Repositories?

A repository is an online (or local) location that stores packages.

- Provides trusted and verified packages.
- Helps you get updates and security patches.
- Each Linux distribution has its own repositories.

Examples:

- Ubuntu/Debian: Official APT repos
- RHEL/CentOS/Fedora: Official DNF repos
- Third-party repos: e.g. Docker, Nginx, etc.

What are Dependencies?

A package may require other packages (libraries or tools) to work. These are called dependencies.

- Package managers automatically install missing dependencies.
- Removing a package may also remove dependent packages.
- Dependencies ensure software works correctly.

Example:

nginx depends on libraries like libpcre, zlib, openssl, etc.

APT (Ubuntu/Debian)

Common APT commands:

```
$ sudo apt update # update repo list
$ sudo apt upgrade # upgrade packages
$ sudo apt install nginx # install
$ sudo apt remove nginx # remove
$ sudo apt search nginx # search
$ sudo apt show nginx # package info
```

Use **apt** on Ubuntu, Debian and derivatives (e.g. Linux Mint).

DNF (RHEL/CentOS/Fedora)

Common DNF commands:

```
$ sudo dnf check-update
$ sudo dnf upgrade
$ sudo dnf install nginx
$ sudo dnf remove nginx
$ sudo dnf search nginx
$ sudo dnf info nginx
```

Use **dnf** on RHEL, CentOS, Fedora and other RPM-based distributions (like Rocky Linux, AlmaLinux).

RPM (Red Hat Package Manager)

Common RPM commands:

```
$ rpm -ivh package.rpm # install
$ rpm -e package # remove
$ rpm -qa | grep nginx # list installed
$ rpm -q nginx # check installed
$ rpm -qi nginx # package info
$ rpm -ql nginx # list files
```

Use **rpm** to work directly with **.rpm** files. Dependency handling is easier with **dnf**.

Updating Packages

Keep your system up to date for security and stability.

```
# Ubuntu/Debian
$ sudo apt update && sudo apt upgrade

# RHEL/CentOS/Fedora
$ sudo dnf upgrade
```

Installing and Removing Software

Install, remove and check packages.

```
# Install
$ sudo apt install git
$ sudo dnf install git

# Remove
$ sudo apt remove git
$ sudo dnf remove git
```

Search and Get Information

Find packages and view details.

```
$ apt search nginx # Ubuntu/Debian
$ dnf search nginx # RHEL/Fedora
$ apt show nginx # package details
$ dnf info nginx # package details
$ rpm -qi nginx # package info
```

Why Production Upgrades Require Caution



- Upgrades can introduce breaking changes.
- Services may stop working after an update.
- Configurations may be overwritten.
- New versions may have different dependencies.
- Always test in a non-production environment first.
- Have a backup and rollback plan.

Best Practices

- ✓ Keep repositories trusted and updated.
- ✓ Read release notes before upgrading.
- ✓ Test changes in a staging environment.
- ✓ Use package version pinning if needed.
- ✓ Schedule upgrades during maintenance windows.
- ✓ Monitor services after updates.
- ✓ Keep a rollback plan.

Key Takeaway

“Packages make Linux powerful, but in production environments, always upgrade with a plan, not just a command.”



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Understanding disks, partitions, filesystems and mount points is essential for system administration, troubleshooting and performance. Storage issues are one of the most common causes of application failures.

Block Devices

Block devices are devices that store data in fixed-size blocks (e.g. disks, SSDs, virtual disks).

```
$ lsblk
NAME MAJ:MIN SIZE TYPE
sda      8:0    100G disk
├─sda1    8:1      1G part
├─sda2    8:2     50G part
└─sda3    8:3     49G part
```

Common block devices:
`/dev/sda`, `/dev/nvme0n1`, `/dev/vda`
(names may vary).

Partitions

A disk can be split into partitions. Each partition can have its own filesystem.

```
$ lsblk -f
NAME FSTYPE LABEL MOUNTPOINT
sda
├─sda1 xfs      /boot
├─sda2 xfs      /
└─sda3 ext4    /data
```

Partitions allow you to organize data and use different filesystems.

Filesystems

A filesystem stores and organizes files on a partition. Common filesystems include ext4, xfs, btrfs.

```
$ df -T
Filesystem Type Size Used Avail
/dev/sda2 xfs 50G 10G 40G
/dev/sda3 ext4 49G 5G 44G
tmpfs tmpfs 2.0G 0 2.0G
```

Choose the right filesystem based on your use case (e.g. xfs for large files, ext4 for general use).

Mount Points

A mount point is a directory where a filesystem is attached and made accessible.

```
$ mount | head
/dev/sda2 on / type xfs (rw,relatime)
/dev/sda1 on /boot type xfs (rw,relatime)
/dev/sda3 on /data type ext4 (rw,relatime)
```

Use the `mount` command to attach filesystems. Use `umount` to detach.

Useful Commands

Key commands for storage and disk usage.

```
$ lsblk # list block devices
$ df -h # disk space usage
$ du -sh /var # directory size
$ mount # show mounted filesystems
$ umount /data # unmount a filesystem
```

/etc/fstab

The `/etc/fstab` file defines filesystems that are automatically mounted at boot.

```
# <device> <mount point> <type> <options>
/dev/sda2 / xfs defaults 0 0
/dev/sda1 /boot xfs defaults 0 0
/dev/sda3 /data ext4 defaults 0 0
```

Always test changes with `mount -a` before rebooting.

Inodes

An inode stores metadata about a file (name, permissions, size, owner, etc.). Each file uses one inode.

```
$ df -i
Filesystem Inodes IUsed IFree IUse%
/dev/sda2 3276800 123456 3153344 4%
```

You can run out of inodes even if you have disk space. This usually happens with many small files (e.g. logs).

Troubleshooting "No space left on device"

This error can be caused by:

- Disk space full
- Inodes full
- Large log files
- Deleted files still open by a process



Steps to troubleshoot:

- 1 Check disk space: `df -h`
- 2 Check inodes: `df -i`
- 3 Find large directories: `du -sh /*`
- 4 Check large files: `find / -type f -size +100M`
- 5 Check open deleted files: `lssof | grep deleted`
- 6 Clean up logs and temporary files
- 7 Consider increasing disk size or inodes

Key Takeaways

- ✓ Understand block devices, partitions and filesystems.
- ✓ Use `lsblk`, `df -h` and `du -sh` to check storage usage.
- ✓ Know your mount points and `/etc/fstab`.
- ✓ Monitor both disk space and inodes.
- ✓ Investigate the cause before making changes.
- ✓ Clean up old logs and unused files regularly.
- ✓ Avoid running out of space by setting up monitoring and alerts.
- ✓ Storage issues can cause application failures, so always troubleshoot carefully.

“CHECK BEFORE YOU CHANGE. DATA MATTERS.”



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQT

Memory and CPU are critical system resources. Learn how to monitor usage, understand key metrics and troubleshoot high resource utilization before it affects your systems and applications.

RAM (Memory)

RAM is volatile memory used by running processes. More RAM allows your system to run more applications.

```
$ free -h
              total used free shared buff/cache available
Mem:  7.7G  4.2G  1.9G  256M  1.6G  3.1G
Swap:  2.0G   0B  2.0G
```

Use `free -h` to view memory usage in human readable format.

Swap

Swap is disk space used as virtual memory when RAM is full. It is slower than RAM but helps prevent the system from running out of memory.

```
$ free -h
              total used free
Mem:  7.7G  4.2G  1.9G
Swap:  2.0G   0B  2.0G
```

Swap is used when RAM is exhausted. Excessive swapping can make the system slow.

Load Average

Load average shows the average number of processes waiting for CPU or I/O over 1, 5 and 15 minutes.

```
$ uptime
12:34:56 up 3 days, 2:14, 2 users
load average: 0.48, 0.52, 0.58
```

A load average close to the number of CPU cores is normal. A higher value may indicate resource pressure.

CPU Utilization

CPU utilization shows how busy your CPU is. High CPU usage can affect application performance.

```
$ top -bn1 | head -5
top - 12:34:56 up 3 days, 2:14
Tasks: 184 total, 2 running
%Cpu(s): 12.5 us, 5.3 sy, 0.0 ni,
MiB Mem : 7851.9 total, 4190.2 used
MiB Swap: 2048.0 total, 0.0 used
```

Use `top` to see real time CPU usage and running processes.

Useful Commands

Key commands to check memory and CPU usage.

```
$ free -h # show memory usage
$ uptime # show load average
$ top     # real time monitoring
$ vmstat 1 5 # memory, CPU, I/O stats
$ nproc   # number of CPU cores
$ lscpu    # detailed CPU information
```

Use these commands together to get a complete picture.

vmstat

vmstat shows memory, CPU, paging, block I/O and more.

```
$ vmstat 1 3
procs -----memory----- --swap-- --io--
r b swpd free buff cache si so bi bo
1 0 0 2103456 123456 987654 0 0 12 34
0 0 0 2097152 123456 987800 0 0 10 20
0 0 0 2089344 123456 988100 0 0 8 18
```

Run `vmstat 1 5` to see 5 updates every 1 second.

Understanding Resource Pressure

Resource pressure happens when demand is higher than available resources.

- High CPU usage: system is busy processing.
- High memory usage: system may start using swap.
- High load average: more processes waiting.
- High I/O wait: system is slow due to disk I/O.
- Check all metrics together, not just one.

High CPU Troubleshooting

Steps to investigate high CPU usage.

- 1 Use `top` or `htop` to find the process using CPU.
- 2 Check if it is expected (e.g. batch job).
- 3 Investigate with `ps`, `strace` or application logs.
- 4 Look for runaway processes or high loops.
- 5 Consider tuning, scaling or fixing code.

High Memory Troubleshooting

Steps to investigate high memory usage.

- 1 Use `free -h` to check memory and swap.
- 2 Use `top` to find the process using memory (RES column).
- 3 Check for memory leaks.
- 4 Look at application logs.
- 5 Consider increasing RAM, optimizing the application or restarting the service (if safe).

Key Takeaways

- ✓ Monitor memory, CPU and load average regularly.
- ✓ Understand normal behavior for your systems.
- ✓ Investigate high usage before it causes outages.
- ✓ Use the right tools for the right problem.
- ✓ Plan capacity and set up monitoring and alerts.

“KNOW YOUR RESOURCES BEFORE THEY RUN OUT.”



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

Networking is fundamental to Linux and DevOps. This pocket toolkit covers the essential commands to check interfaces, IP addresses, routes, DNS, open ports and basic connectivity troubleshooting.

Network Interfaces

Network interfaces connect your Linux system to networks (physical or virtual).

```
$ ip addr
1: lo: <LOOPBACK> mtu 65536
   inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST>
   inet 192.168.1.10/24 brd 192.168.1.255
```

Shows all network interfaces and their IP addresses.

IP Addresses

View and manage IP addresses on your system.

```
$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP>
   inet 192.168.1.10/24
   inet6 fe80::a00:27ff:fe4e:66a1/64
```

Shows details for a specific interface. Use **ip addr** to see IPv4 and IPv6 addresses.

Routes

Routes tell your system where to send network traffic.

```
$ ip route
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel
10.0.0.0/24 via 192.168.1.100 dev eth0
```

Shows the routing table. The default route (**default via**) is where traffic goes when no specific route is found.

DNS (Domain Name System)

DNS converts domain names to IP addresses.

```
$ dig veriqta.com
;; ANSWER SECTION:
veriqta.com.      300    IN     A
104.21.16.1
```

Look up DNS records using **dig**. Common records: A (IPv4), AAAA (IPv6), CNAME, MX, TXT.

Ports

Ports allow different services to run on the same server.

```
$ ss -tln
Netid  State  Local Address:Port
tcp    LISTEN 0.0.0.0:22
tcp    LISTEN 0.0.0.0:80
tcp    LISTEN 127.0.0.1:3306
udp    UNCONN 0.0.0.0:53
```

Shows open ports and listening services. **-t** = TCP, **-u** = UDP, **-l** = listening, **-n** = numeric addresses.

ping

Test basic network connectivity.

```
$ ping -c 4 google.com
PING google.com (142.250.72.14)
56(84) bytes of data:
4 packets transmitted, 4 received,
0% packet loss, time 3006ms
rtt min/avg/max = 12.3/14.6/18.9 ms
```

Checks if a host is reachable. Use **-c** to specify the number of packets.

curl

Test HTTP/HTTPS services.

```
$ curl -I https://veriqta.com
HTTP/2 200
server: cloudflare
content-type: text/html
content-length: 1234
```

Sends an HTTP request. **-I** shows only headers. Use **curl** to test APIs and web services.

traceroute

Show the path your traffic takes to a destination.

```
$ traceroute google.com
1  192.168.1.1      1.1 ms
2  10.0.0.1        5.4 ms
3  72.14.213.1     12.6 ms
4  142.250.72.14   14.8 ms
```

Shows each hop between your system and the destination. Useful for finding where connectivity issues occur.

Basic Connectivity Troubleshooting

Steps to troubleshoot network issues.

- 1 Check interface is up: **ip addr**
- 2 Check IP address: **ip addr**
- 3 Check default route: **ip route**
- 4 Test local connectivity: **ping 127.0.0.1**
- 5 Test external connectivity: **ping 8.8.8.8**
- 6 Test DNS resolution: **dig google.com**
- 7 Check open ports: **ss -tln**
- 8 Test a service: **curl -I <url>**
- 9 Trace the path: **traceroute <host>**

COMMON ISSUES

- No IP address (interface down).
- Cannot reach gateway (routing issue).
- DNS not resolving (check `/etc/resolv.conf`).
- Port not open (service not running or firewall).
- High latency or packet loss (network or ISP issue).

KEY TAKEAWAYS

- ✓ Know your interfaces, IP addresses and routes.
- ✓ Use the right tool for the right problem.
- ✓ Always check DNS and connectivity.
- ✓ Understand what open ports mean.
- ✓ Follow a step-by-step approach.
- ✓ Networking issues are often outside your server.

“A CONNECTED SYSTEM IS A PRODUCTIVE SYSTEM.”



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

SSH (Secure Shell) allows you to securely connect to and manage remote servers. It uses encrypted communication and is the standard method for remote Linux administration.

What is SSH?

SSH is a secure network protocol used to access remote systems, execute commands and transfer files. It encrypts all data between your machine and the remote server.

Common uses:

- ✓ Remote server administration
- ✓ Secure file transfers
- ✓ Tunnel other network traffic
- ✓ Automate server access (scripts)

Connect with SSH

Basic SSH connection:

```
$ ssh user@server
# example
$ ssh ubuntu@192.168.1.10
$ ssh ec2-user@54.210.10.5
```

Useful options:

```
$ ssh -p 2222 user@server # custom port
$ ssh -i key.pem user@server # use key
$ ssh -v user@server # verbose output
```

Public / Private Key Pairs

SSH uses a public/private key pair for a more secure and passwordless login.

```
$ ssh-keygen -t ed25519 -C "your_email"
# creates a new key pair
# private key: ~/.ssh/id_ed25519
# public key: ~/.ssh/id_ed25519.pub
```

Keep your private key secret.
Share only the public key with the server
(add it to `~/.ssh/authorized_keys`).

The ~/.ssh/ Directory

Your SSH keys and configuration files are stored in the `~/.ssh/` directory.

```
$ ls -la ~/.ssh/
id_ed25519      # private key
id_ed25519.pub  # public key
authorized_keys # keys allowed to login
known_hosts     # known server fingerprints
config         # optional SSH configuration
```

Keep this directory secure with permissions 700: `chmod 700 ~/.ssh`

authorized_keys

The server stores your public key in `~/.ssh/authorized_keys` to allow access.

```
$ cat ~/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...
user@laptop
```

You can add multiple public keys for different users or devices.

known_hosts

The `~/.ssh/known_hosts` file stores the fingerprints of servers you have connected to, preventing man-in-the-middle attacks.

```
$ cat ~/.ssh/known_hosts
192.168.1.10 ssh-ed25519 AAAAC3N...
github.com  ssh-rsa AAAAB3Nza...
```

If a server's key changes, SSH will warn you. Verify the change before accepting it.

scp – Secure Copy

Use `scp` to securely copy files between your local machine and a remote server.

```
# copy local file to remote server
$ scp file.txt user@server:/home/user/
# copy file from remote server
$ scp user@server:/var/log/syslog ./
# copy a directory
$ scp -r myfolder/ user@server:/home/user/
```

SSH Key Permissions

Your private key must have strict permissions.

```
$ chmod 600 ~/.ssh/id_ed25519
$ chmod 700 ~/.ssh/
$ chmod 644 ~/.ssh/id_ed25519.pub
```

 If permissions are too open, SSH may refuse to use the key.

Authentication Failures

Common reasons for SSH login failures:

- Wrong username or server address
- Wrong private key or key not found
- Incorrect key permissions
- Public key not in `authorized_keys`
- SSH service not running on server
- Firewall blocking the port
- Trying the wrong port (not 22)

Why Private Keys Must Never Be Shared



- A private key gives full access to your server.
- Anyone with your private key can log in as you.
- It cannot be easily revoked if leaked.
- Always keep your private key secure.
- Use a passphrase for additional protection.

Best Practices

- ✓ Use key-based authentication.
- ✓ Keep private keys secure and never share them.
- ✓ Use strong passphrases.
- ✓ Regularly review and remove unused keys.
- ✓ Use non-default SSH ports where appropriate.
- ✓ Monitor authentication logs (`/var/log/auth.log`).

Common Useful Commands

```
ssh user@server # connect
scp file user@server:/path # copy file
ssh-keygen -t ed25519 # generate key
cat ~/.ssh/authorized_keys # view keys
cat ~/.ssh/known_hosts # view known hosts
chmod 600 ~/.ssh/id_ed25519 # set key permissions
ssh -p 2222 user@server # use custom port
ssh -v user@server # verbose output
```

LEARN
PRACTICE
GROW
WITH VERIQT



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

Environment variables, shell features and basic commands are essential for a productive Linux experience. They control how your shell behaves, store important configuration and are used by applications and CI/CD systems.

Shell Variables

Shell variables store data in your current shell session. They are simple key-value pairs.

```
$ NAME=VeriQta
$ echo $NAME
VeriQta
```

No spaces around = when assigning values. Use \$ to access the variable.

Environment Variables

Environment variables are available to your shell and to child processes (applications).

```
$ export APP_ENV=production
$ echo $APP_ENV
production
```

Use **export** to make a variable available to applications and scripts.

echo Command

echo prints text and the value of variables.

```
$ echo "Hello Linux"
Hello Linux
$ echo $USER
veriQta
```

Use **echo** to display messages, variable values and for debugging in scripts.

env Command

env shows all environment variables in your current session.

```
$ env | less
$ env | grep PATH
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
```

Use **grep** to find specific variables.

export Command

The **export** command creates or updates environment variables.

```
$ export API_URL=https://example.com
$ echo $API_URL
https://example.com
```

Exported variables are inherited by child processes.

\$PATH

\$PATH tells Linux where to look for executable files.

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

Add a directory to **PATH**:

```
$ export PATH=$PATH:/opt/tools
```

.bashrc

The **.bashrc** file is used to set variables, aliases and shell preferences for your user.

```
$ nano ~/.bashrc
# Add your variables and aliases here
export PATH=$PATH:/opt/tools
alias ll='ls -la'
```

After editing, reload it with:
\$ source ~/.bashrc

Command History

Bash keeps a history of the commands you run.

```
$ history | tail
998 kubectl get pods
999 git status
1000 docker ps
```

Useful shortcuts:

- **history** → show command history
- **!!** → run last command
- **!n** → run command number n

Aliases

Aliases let you create shortcuts for commands.

```
$ alias ll='ls -la'
$ alias k='kubectl'
$ ll
total 12
drwxr-xr-x 3 veriQta veriQta 4096 Sep 21 10:00 .
```

Add aliases to **~/.bashrc** to make them permanent.

Exit Codes

Every command returns an exit code. 0 means success. Non-zero means an error.

```
$ ls /nonexistent-folder
ls: cannot access '/nonexistent-folder'
$ echo $?
2
```

Use **echo \$?** to check the exit code of the last command.

Why Environment Variables Matter

Environment variables are used by:

- Applications (configuration, database credentials, API keys).
- CI/CD pipelines (build settings, secrets, deployment configuration).
- Scripting and automation (dynamic values).
- Different environments (dev, test, staging, production).

They make systems flexible, secure and easy to manage.

Key Takeaways

- ✓ Understand shell vs environment variables.
- ✓ Use **export** for variables needed by applications.
- ✓ Know your **\$PATH**.
- ✓ Store customizations in **~/.bashrc**.
- ✓ Use command history and aliases to be more productive.
- ✓ Always check exit codes with **echo \$?** (0 = success).
- ✓ Environment variables are critical for applications and CI/CD systems.



“SMALL COMMANDS. BIG OPPORTUNITIES.”



@veriQta



github.com/veriQta



veriQta



t.me/veriQta



veriQta



@veriQta

LEARN
PRACTICE
GROW
WITH VERIQT

Linux is the foundation of DevOps. It powers automation, CI/CD, containers, Kubernetes and modern infrastructure. Learn the key concepts and commands to use Linux effectively in real-world DevOps environments.

Bash Scripts

Bash scripts let you automate tasks, reduce manual work and make your infrastructure reproducible.

```
$ cat greet.sh
#!/bin/bash
echo "Hello DevOps!"
echo "Today is $(date)"
```

Save the file, make it executable and run it using `./greet.sh`.

Shebang

The shebang tells Linux which interpreter to use to run your script.

```
#!/bin/bash
#!/usr/bin/env bash
#!/bin/sh
```

Use `#!/bin/bash` at the top of your script to ensure it runs with Bash.

Variables

Variables store data that can be used in your script.

```
NAME="VeriQTA"
ENV="production"
echo "Name: $NAME"
echo "Environment: $ENV"
```

Use `$` to access variables. Use quotes for values with spaces.

Conditions

Use conditions to make decisions in your scripts.

```
if [ "$ENV" = "production" ]; then
  echo "Production environment"
else
  echo "Not production"
fi
```

Common operators: `=`, `!=`, `-z` (empty), `-n` (not empty).

Loops

Loops help you repeat tasks.

```
# For loop
for i in 1 2 3; do
  echo "Number: $i"
done

# While loop
count=1
while [ $count -le 3 ]; do
  echo "Count: $count"
  ((count++))
done
```

Executable Scripts

Make your script executable and run it.

```
$ chmod +x deploy.sh
$ ./deploy.sh
```

You can also run scripts with:

```
$ bash deploy.sh
$ sh deploy.sh
```

Cron

Cron runs scripts on a schedule.

```
# Edit crontab
$ crontab -e
# Run a script every day at 2 AM
0 2 * * * /home/user/backup.sh
```

Use `cron` for backups, cleanups and routine automation.

Environment Configuration

Use environment variables and config files to manage settings.

```
export APP_ENV=production
export API_URL=https://api.example.com
export PATH=$PATH:/opt/scripts
```

Add variables to `~/.bashrc` or `~/.profile` to make them persistent.

Linux in CI/CD Runners

Most CI/CD runners use Linux. You can run scripts, install tools and automate build, test and deployment steps.

```
# Example CI step (GitHub Actions)
- name: Run deployment script
  run: |
    chmod +x deploy.sh
    ./deploy.sh
```

Linux Hosts Running Docker

Docker containers run on a Linux host. Understand the underlying Linux system to manage storage, networking and resources.

```
$ docker ps
$ docker logs <container_id>
$ docker exec -it <container_id> bash
```

Linux skills help you troubleshoot Docker issues.

Linux Underneath Kubernetes Nodes

Kubernetes runs on Linux nodes. You need Linux skills to manage the nodes, check services and troubleshoot issues.

```
$ systemctl status kubelet
$ journalctl -u kubelet
$ crictl ps
$ ip addr
$ df -h
```

A strong Linux foundation is essential for Kubernetes.

Infrastructure Automation

Linux is the base for automation tools like Ansible, Terraform and cloud CLIs.

```
# Ansible example
ansible all -m ping

# Terraform example
terraform init
terraform apply
```

Linux enables you to automate and manage infrastructure at scale.



Key Takeaways

- ✓ Linux is the foundation of DevOps.
- ✓ Automation starts with shell scripts.
- ✓ Linux powers CI/CD, containers and Kubernetes.
- ✓ Strong Linux skills make you a better DevOps engineer.



**"AUTOMATE TODAY
A MORE RELIABLE TOMORROW."**

LEARN
PRACTICE
GROW
WITH VERIQT



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta

A simple, repeatable troubleshooting process for junior engineers. Follow the steps, use the core commands and always make decisions based on evidence. Do not make random changes.

1. Troubleshooting Sequence

- 1 What changed?
- 2 Check system state
- 3 Check service
- 4 Read logs
- 5 Check CPU / memory / disk
- 6 Check network / ports
- 7 Check permissions
- 8 Form hypothesis
- 9 Apply smallest safe fix
- 10 Verify

2. Core Commands

Command	What it does
uptime	Shows how long the system has been running and load average.
top	Shows real-time CPU and memory usage.
free -h	Displays memory and swap usage in human readable format.
df -h	Shows disk space usage.
ps aux	Lists running processes.
systemctl status	Shows the status of a service.
journalctl	Views system and service logs.
ip addr	Shows network interfaces and IP addresses.
ip route	Shows the routing table.
ss -tulpn	Shows open ports and listening services.
curl	Tests HTTP/HTTPS connectivity.
dig	Queries DNS records.
grep	Searches text for patterns.
find	Finds files and directories.

3. Helpful Examples

```
$ uptime           # system load
$ top              # live resource usage
$ free -h          # memory usage
$ df -h            # disk usage
$ ps aux           # running processes
$ systemctl status nginx # check service
$ journalctl -u nginx # view service logs
$ ip addr          # IP addresses
$ ip route         # routing table
$ ss -tulpn        # open ports
$ curl -I https://example.com # test HTTP
$ dig example.com  # DNS lookup
$ grep error /var/log/syslog # search logs
$ find /var -name "*.log" # find files
```

Combine commands to get a full picture.
Example:

```
$ systemctl status nginx
$ journalctl -u nginx --since "1 hour ago"
$ ss -tulpn | grep nginx
```

4. What to Check at Each Step

- 1 What changed? (deploy, config, package, user, network, hardware, etc.)
- 2 Check system state (uptime, load, errors).
- 3 Check service (is it running, failed, restarting?).
- 4 Read logs (system logs, application logs).
- 5 Check CPU / memory / disk (is the server under pressure?).
- 6 Check network / ports (is it reachable? are ports open?).
- 7 Check permissions (file, directory, user, sudo).
- 8 Form hypothesis (most likely cause).
- 9 Apply smallest safe fix (least risky change).
- 10 Verify (confirm the issue is resolved and monitor).

5. Common Issues and How to Check

Issue	Commands to Use
High CPU usage	top, ps aux
High memory usage	free -h, top
Disk full	df -h, du -sh *
Service not running	systemctl status <service>
Service failing	journalctl -u <service>
Port not open	ss -tulpn, curl
Network issue	ip addr, ip route, ping, curl
DNS issue	dig, nslookup
Permission denied	ls -l, id, namei
Process not found	ps aux, grep
File not found	find, ls
Slow application	top, journalctl, df -h, ss

6. Golden Rules

- ✓ Always check the evidence first.
- ✓ Do not randomly restart services.
- ✓ Do not delete files to "fix" issues.
- ✓ Do not change permissions blindly.
- ✓ Do not use sudo until you understand what you are doing.
- ✓ Make the smallest safe change.
- ✓ Verify the fix and monitor.
- ✓ Document what you found and what you changed.
- ✓ Learn from every incident.



**DO NOT GUESS.
DO NOT MAKE RANDOM CHANGES.
LET THE EVIDENCE GUIDE YOU.**

**“A CALM MIND, A CLEAR PROCESS
AND THE RIGHT COMMANDS CAN SOLVE
MOST LINUX PROBLEMS.”**



KEY TAKEAWAY

Troubleshooting is not about luck. It is a skill. Follow the process, use the right tools and always work with evidence.

LEARN
PRACTICE
GROW
WITH VERIQTA



@veriqta



github.com/veriqta



veriqta



t.me/veriqta



veriqta



@veriqta